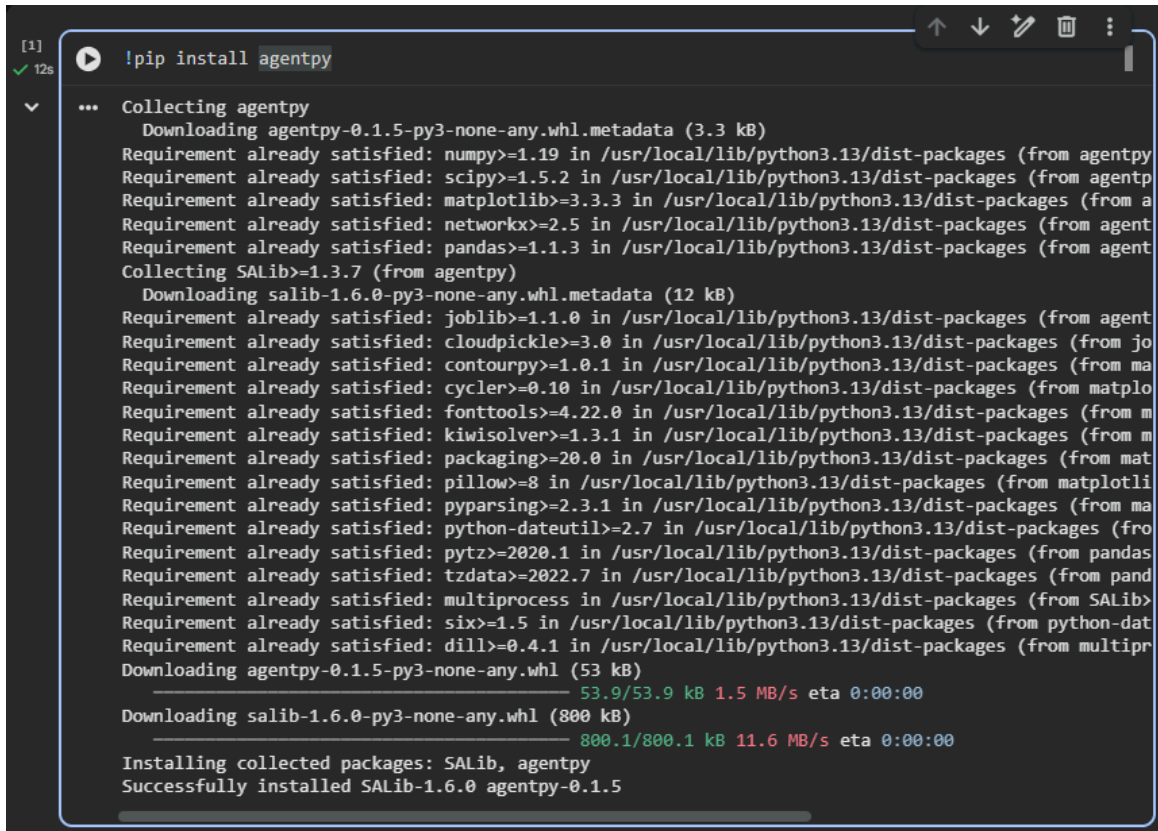


1. Run the Code:
 - a. Installing the libraries needed.



```
[1] ✓ 12s !pip install agentpy

... Collecting agentpy
  Downloading agentpy-0.1.5-py3-none-any.whl.metadata (3.3 kB)
Requirement already satisfied: numpy>=1.19 in /usr/local/lib/python3.13/dist-packages (from agentpy)
Requirement already satisfied: scipy>=1.5.2 in /usr/local/lib/python3.13/dist-packages (from agentpy)
Requirement already satisfied: matplotlib>=3.3.3 in /usr/local/lib/python3.13/dist-packages (from agentpy)
Requirement already satisfied: networkx>=2.5 in /usr/local/lib/python3.13/dist-packages (from agentpy)
Requirement already satisfied: pandas>=1.1.3 in /usr/local/lib/python3.13/dist-packages (from agentpy)
Collecting SALib>=1.3.7 (from agentpy)
  Downloading salib-1.6.0-py3-none-any.whl.metadata (12 kB)
Requirement already satisfied: joblib>=1.1.0 in /usr/local/lib/python3.13/dist-packages (from SALib)
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.13/dist-packages (from SALib)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from SALib)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from SALib)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from SALib)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from SALib)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from SALib)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packages (from SALib)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from SALib)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.13/dist-packages (from SALib)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from SALib)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from SALib)
Requirement already satisfied: multiprocessing in /usr/local/lib/python3.13/dist-packages (from SALib)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from SALib)
Requirement already satisfied: dill>=0.4.1 in /usr/local/lib/python3.13/dist-packages (from SALib)
  Downloading agentpy-0.1.5-py3-none-any.whl (53 kB)
    53.9/53.9 kB 1.5 MB/s eta 0:00:00
  Downloading salib-1.6.0-py3-none-any.whl (800 kB)
    800.1/800.1 kB 11.6 MB/s eta 0:00:00

Installing collected packages: SALib, agentpy
Successfully installed SALib-1.6.0 agentpy-0.1.5
```

b. Copy pasting the code

```
[ ]

import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# ----- Get user input from console -----
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# ----- Agent Definition -----
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# ----- Model Definition -----
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                             random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
```

```

[ ] ▶ def step(self):
        for agent in self.agents:
            agent.step()

# ----- Parameters from user input -----
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# ----- Run Model -----
model = RandomWalkModel(parameters)
model.setup()

# ----- Interactive Animation -----
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200) # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()

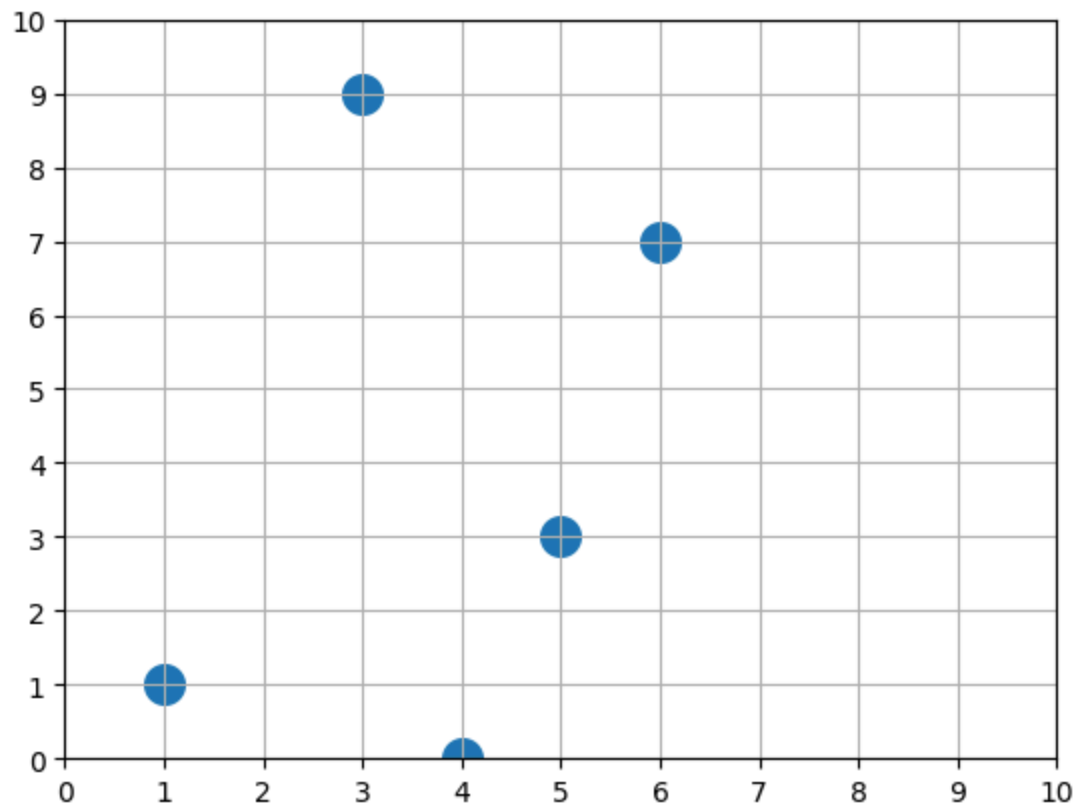
```

c. Running the code with the given data set

```

... Enter number of agents: 5
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 

```



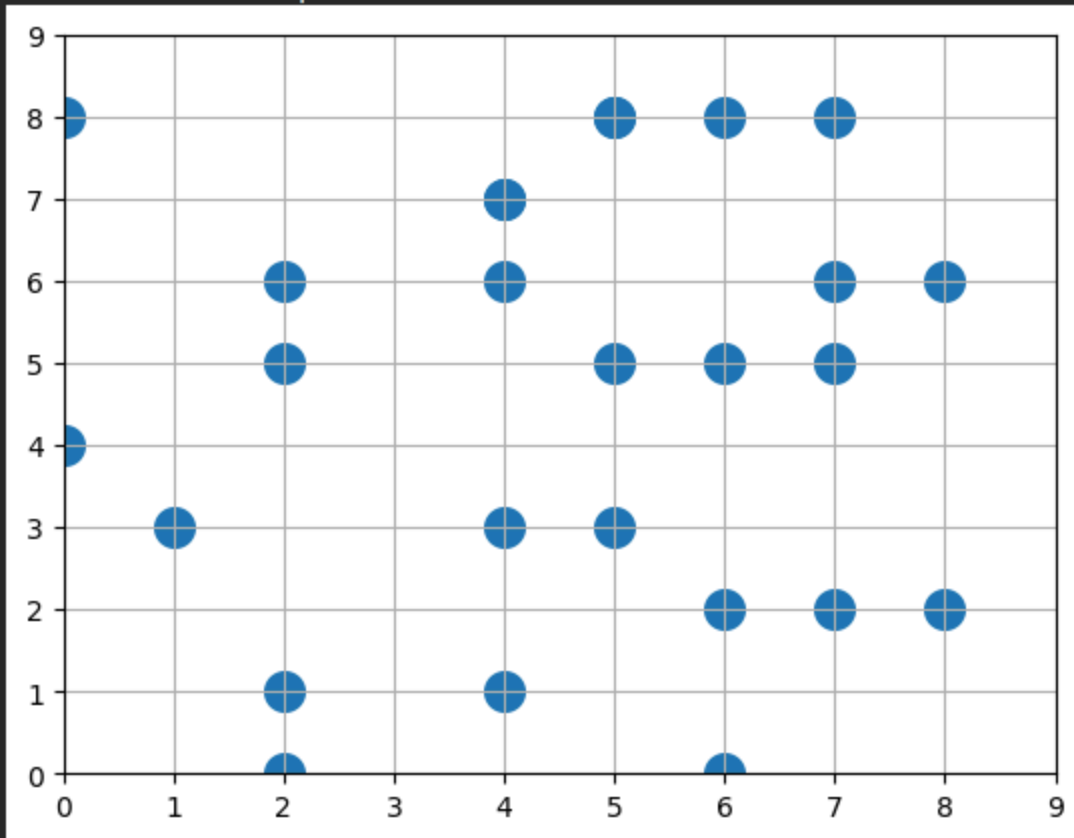
2. Experimenting the code

UNMODIFIED CODE

a. Exp-01

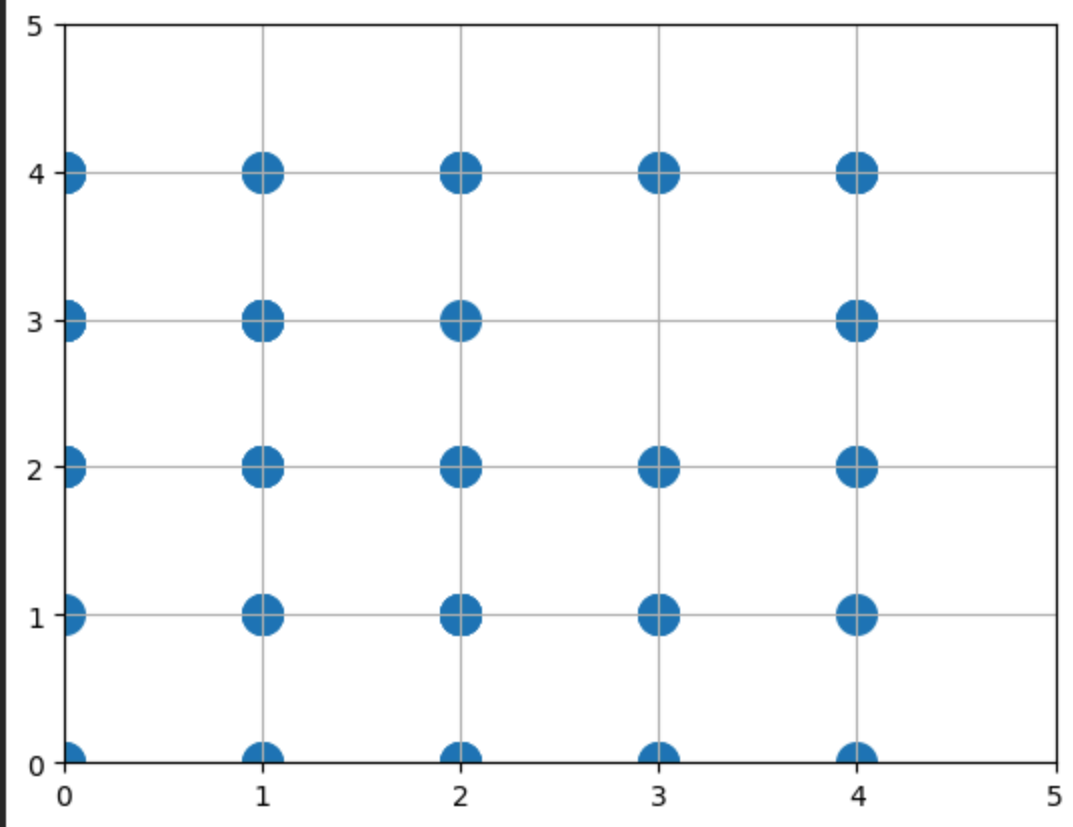
- i. Enter number of agents: 26
- ii. Enter grid size: 9
- iii. Enter number of steps: 50

```
... Enter number of agents: 26
Enter grid size (e.g., 10 for 10x10): 9
Enter number of steps: 50
```



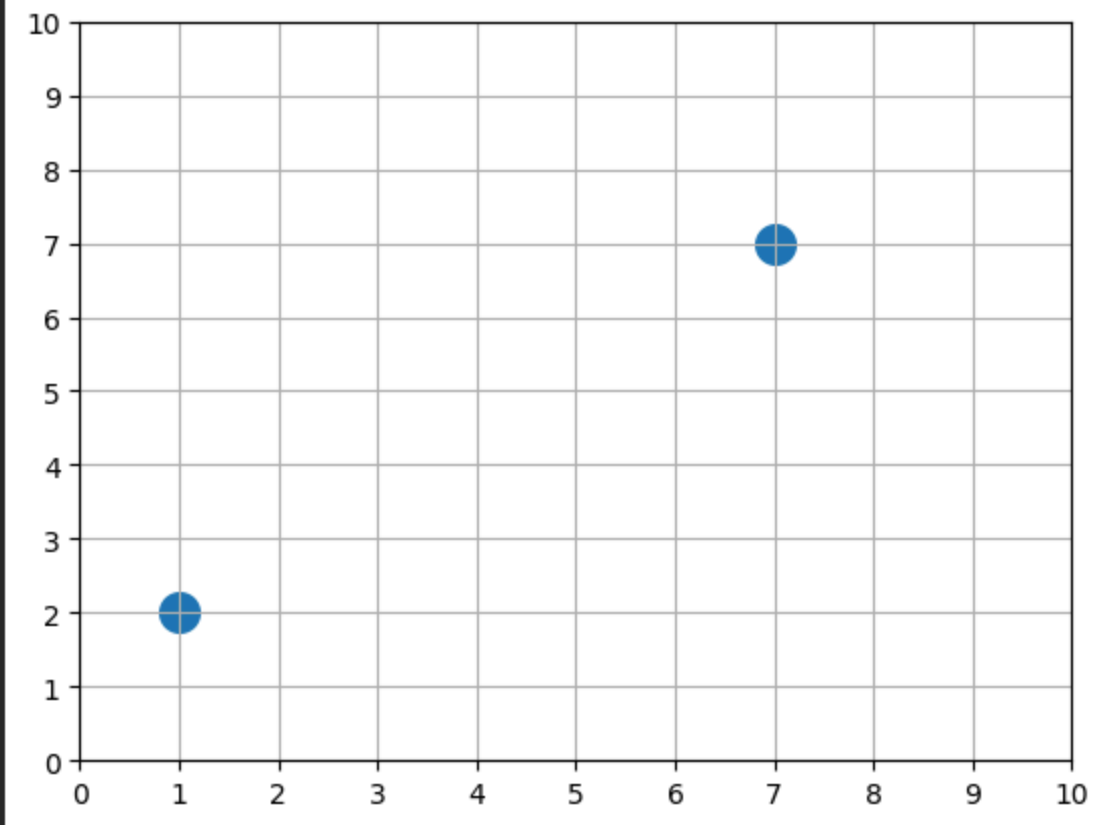
- b. Exp-02
- Enter number of agents: 81
 - Enter grid size: 5
 - Enter number of steps: 29

```
... Enter number of agents: 81
Enter grid size (e.g., 10 for 10x10): 5
Enter number of steps: 29
```



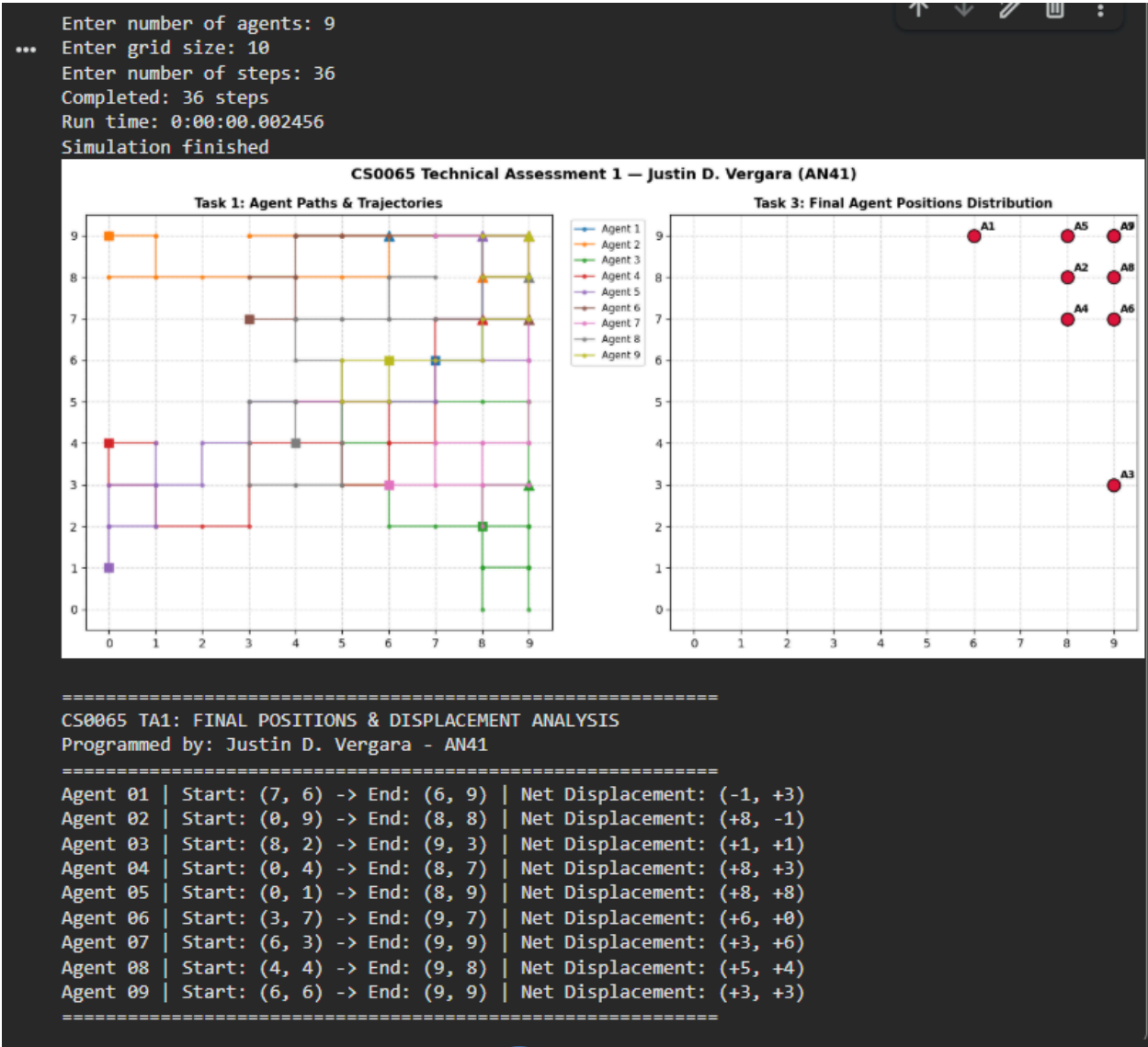
- c. Exp-03
- i. Enter number of agents: 2
 - ii. Enter grid size: 10
 - iii. Enter number of steps: 60

```
... Enter number of agents: 2  
Enter grid size (e.g., 10 for 10x10): 10  
Enter number of steps: 60
```

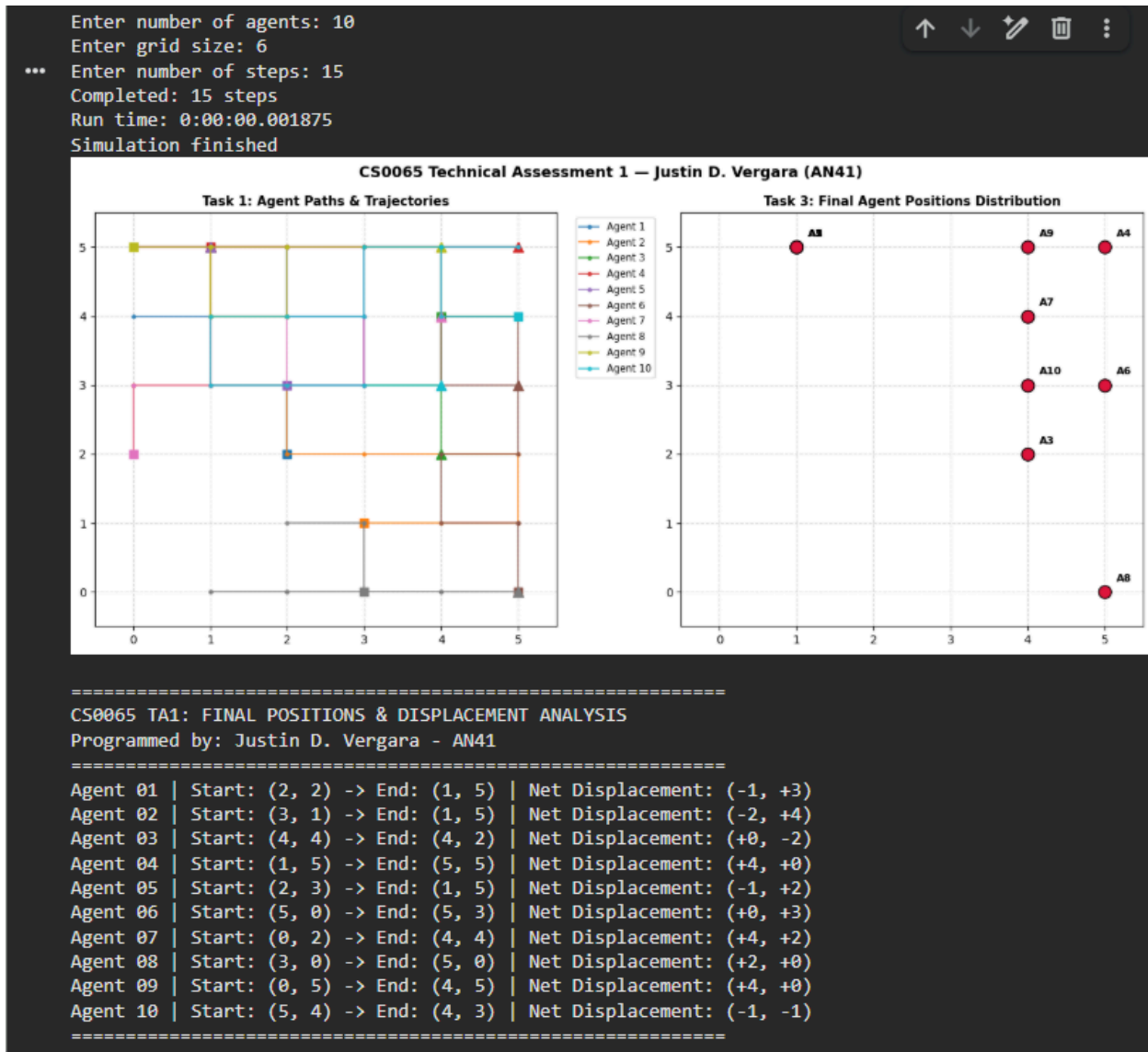


MODIFIED CODE

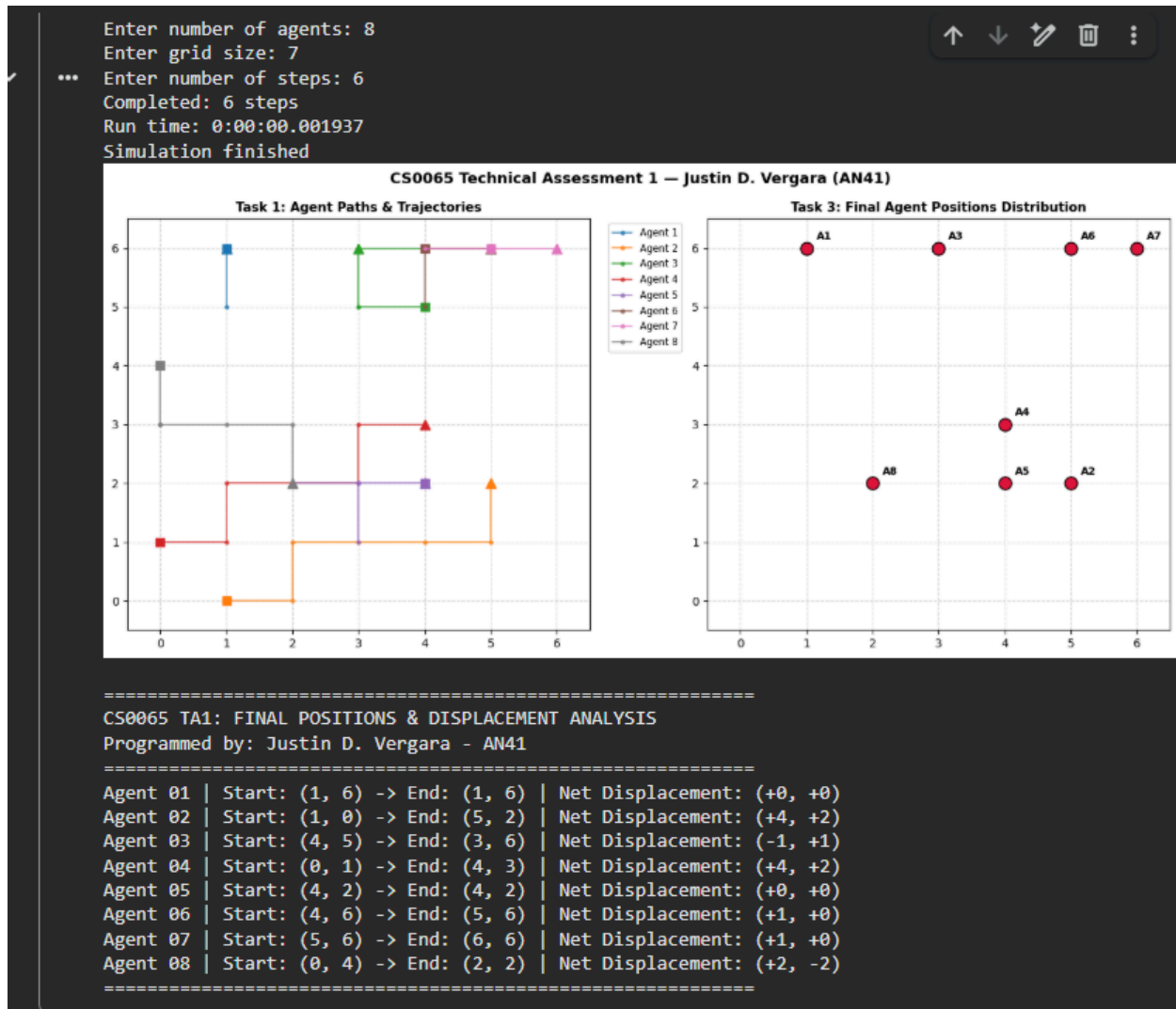
- d. Exp-04
 - i. Enter number of agents: 9
 - ii. Enter grid size: 10
 - iii. Enter number of steps: 36



- e. Exp-05
 - i. Enter number of agents: 10
 - ii. Enter grid size: 6
 - iii. Enter number of steps: 15



- f. Exp-06
 - i. Enter number of agents: 8
 - ii. Enter grid size: 7
 - iii. Enter number of steps: 6



3. Modified Code

```
[10]
✓ 5s
import random
import agentpy as ap
import matplotlib.pyplot as plt

# 1. AGENT CLASS DEFINITION
class SmartWalker(ap.Agent):
    """
    Agent with trajectory tracking and modified directional movement rules.
    """
    def setup(self):
        # Initialize agent at a random coordinate within grid bounds
        self.position = (
            random.randint(0, self.model.p.grid_size[0] - 1),
            random.randint(0, self.model.p.grid_size[1] - 1)
        )
        # Laboratory Task 1: Record and track the complete path
        self.path = [self.position]

    def step(self):
        # Laboratory Task 2: Movement rules modification
        # Biased movement preference: Higher likelihood to step right or up
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
        weights = [0.4, 0.2, 0.3, 0.1] # 40% Right, 20% Left, 30% Up, 10% Down

        move = random.choices(directions, weights=weights, k=1)[0]

        # Bounded Movement: Keep coordinates strictly inside the grid limits
        new_x = max(0, min(self.model.p.grid_size[0] - 1, self.position[0] + move[0]))
        new_y = max(0, min(self.model.p.grid_size[1] - 1, self.position[1] + move[1]))

        self.position = (new_x, new_y)
        self.path.append(self.position)

# 2. MODEL CLASS DEFINITION
class RandomWalkModel(ap.Model):
    """
    Model environment to manage agent population and grid boundaries.
    """
```

[10]

✓ 5s



```
def setup(self):
    self.agents = ap.AgentList(self, self.p.agents, SmartWalker)
    self.grid = ap.Grid(self, self.p.grid_size, torus=False)
    self.grid.add_agents(self.agents)

def step(self):
    self.agents.step()

# 3. USER INPUTS & SIMULATION EXECUTION
# Get inputs dynamically via Colab console prompts
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size: "))
num_steps = int(input("Enter number of steps: "))

parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

model = RandomWalkModel(parameters)
model.run()

# 4. VISUALIZATION: TRAJECTORY PATHS & FINAL POSITIONS
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 6))

# Subplot 1: Agent Trajectory Paths (Task 1)
ax1.set_xlim(-0.5, parameters['grid_size'][0] - 0.5)
ax1.set_ylim(-0.5, parameters['grid_size'][1] - 0.5)
ax1.set_xticks(range(parameters['grid_size'][0]))
ax1.set_yticks(range(parameters['grid_size'][1]))
ax1.grid(True, linestyle="--", alpha=0.5)
ax1.set_title("Task 1: Agent Paths & Trajectories", fontsize=11, fontweight='bold')

colors = plt.cm.tab10.colors
for i, agent in enumerate(model.agents):
    px, py = zip(*agent.path)
    c = colors[i % len(colors)]
    # Path line
```

```

# Path line
ax1.plot(px, py, linestyle='-', marker='o', markersize=3, alpha=0.7, color=c, label=f"Agent {a}")
# Starting location (square) and Ending location (triangle)
ax1.plot(px[0], py[0], marker='s', markersize=7, color=c)
ax1.plot(px[-1], py[-1], marker='^', markersize=8, color=c)

ax1.legend(bbox_to_anchor=(1.03, 1), loc='upper left', fontsize='small')

# Subplot 2: Final Distribution Analysis (Task 3)
ax2.set_xlim(-0.5, parameters['grid_size'][0] - 0.5)
ax2.set_ylim(-0.5, parameters['grid_size'][1] - 0.5)
ax2.set_xticks(range(parameters['grid_size'][0]))
ax2.set_yticks(range(parameters['grid_size'][1]))
ax2.grid(True, linestyle="--", alpha=0.5)
ax2.set_title("Task 3: Final Agent Positions Distribution", fontsize=11, fontweight='bold')

final_x = [agent.path[-1][0] for agent in model.agents]
final_y = [agent.path[-1][1] for agent in model.agents]
ax2.scatter(final_x, final_y, s=120, color='crimson', edgecolors='black', zorder=3)

for agent in model.agents:
    fx, fy = agent.path[-1]
    ax2.annotate(f"A{agent.id}", (fx + 0.15, fy + 0.15), fontsize=9, fontweight='bold')

plt.suptitle("CS0065 Technical Assessment 1 – Justin D. Vergara (AN41)", fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()

# 5. CONSOLE OUTPUT: FINAL POSITION & DISPLACEMENT LOGS
print("\n" + "="*60)
print("CS0065 TA1: FINAL POSITIONS & DISPLACEMENT ANALYSIS")
print("Programmed by: Justin D. Vergara - AN41")
print("="*60)
for agent in model.agents:
    start_pos = agent.path[0]
    end_pos = agent.path[-1]
    dx = end_pos[0] - start_pos[0]
    dy = end_pos[1] - start_pos[1]
    print(f"Agent {agent.id:02d} | Start: {start_pos} -> End: {end_pos} | Net Displacement: ({dx:+}, {dy:+})")
    print("="*60)

```

```

dy = end_pos[1] - start_pos[1]
print(f"Agent {agent.id:02d} | Start: {start_pos} -> End: {end_pos} | Net Displacement: ({dx:+}, {dy:+})")
print("="*60)

```